

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL

**DESENVOLVIMENTO DO PROJETO 8-PUZZLE EM LINGUAGEM C
UTILIZANDO ALGORITMOS DE BUSCA**

FLÁVIO TOMÁS PEÑA VILLA 25006826
ÍTALO FRAGA BOTELHO 25894064
RAFAEL FRANCO LOURENÇON 25897109
TIAGO NODA VON ZUBEN 25018493

CAMPINAS
2025

DESENVOLVIMENTO DO PROJETO 8-PUZZLE EM LINGUAGEM C UTILIZANDO ALGORITMOS DE BUSCA

Relatório de Projeto apresentado como requisito parcial para obtenção de aprovação na disciplina de Estruturas de Dados e Algoritmos, do curso de Ciência de Dados e Inteligência Artificial, da Pontifícia Universidade Católica de Campinas.

Orientadores

Prof. Dr. Denis Mayr Lima Martins – Inteligência Artificial Computacional

Prof. Me. Fernando Soares de Aguiar Neto – Inteligência Artificial Computacional

Prof. Me. Lucas Oliveira Pimenta dos Reis – Programação e Estrutura de Dados

Profa. Dra. Lucia Filomena de Almeida Guimarães – Programação e Estrutura de Dados

CAMPINAS
2025

SUMÁRIO

1	Introdução	2
2	Desenvolvimento do Projeto	2
2.1	Geração e Validação do Tabuleiro	2
2.2	Implementação da Busca em Largura (BFS)	2
2.3	Implementação da Busca em Profundidade Limitada Iterativa (IDDFS)	3
2.4	Otimização da Busca: Poda de Movimentos Reversos	3
2.5	Interface e Experiência do Usuário	3
2.6	Adaptação das Estruturas de Dados	3
2.7	Portabilidade e Compatibilidade (Cross-Platform).....	4
3	Considerações Finais	4
	REFERÊNCIAS	4

1. INTRODUÇÃO

O presente relatório tem como objetivo apresentar o desenvolvimento do projeto 8-Puzzle na linguagem de programação C, contemplando tanto a interação manual do usuário com o tabuleiro quanto a resolução automática por meio de Inteligência Artificial. O trabalho aplicou conceitos de estruturas de dados, grafos e heurísticas, utilizando algoritmos como Busca em Largura (BFS), A*, e Busca em Profundidade Limitada Iterativa (IDDFS). O projeto *8-Puzzle*, uma versão simplificada do *15-Puzzle*, serve como uma excelente plataforma para explorar algoritmos de busca (RUSSELL; NORVIG, 2013).

2. DESENVOLVIMENTO DO PROJETO

Nesta seção são apresentadas as principais dificuldades enfrentadas pela equipe, as soluções adotadas e a justificativa para cada funcionalidade implementada no projeto.

2.1 GERAÇÃO E VALIDAÇÃO DO TABULEIRO

Durante os testes iniciais, observou-se que muitas configurações aleatórias do tabuleiro eram matematicamente insolúveis. Aproximadamente metade das permutações possíveis do *8-Puzzle* apresentam número ímpar de inversões, o que impossibilita a solução do problema. Para evitar travamentos e *loops* infinitos, implementou-se uma função para verificar a paridade das inversões, permitindo somente tabuleiros solúveis (GEEKSFOR-GEEKS, 2025).

2.2 IMPLEMENTAÇÃO DA BUSCA EM LARGURA (BFS)

A Busca em Largura (BFS) foi implementada utilizando uma estrutura de fila. Essa técnica garante a solução ótima, porém apresentou alto consumo de memória em estados profundos, o que resultou em encerramentos inesperados do programa devido ao uso excessivo de RAM.

2.3 IMPLEMENTAÇÃO DA BUSCA EM PROFUNDIDADE LIMITADA ITERATIVA (IDDFS)

A IDDFS combina o baixo consumo de memória da busca em profundidade com a completude da busca em largura. Apesar do maior tempo de execução, mostrou-se mais adequada em cenários nos quais a BFS falhava devido ao consumo elevado de memória.

2.4 OTIMIZAÇÃO DA BUSCA: PODA DE MOVIMENTOS REVERSOS

Foi implementada uma verificação que impede movimentos imediatamente reversos, como mover uma peça para a esquerda e, logo em seguida, mover para a direita. Essa otimização reduz o número de estados analisados, aumentando a eficiência da busca.

2.5 INTERFACE E EXPERIÊNCIA DO USUÁRIO

Para tornar o sistema acessível e intuitivo, foram implementados menus interativos baseados em teclas direcionais, além de mecanismos visuais como barra de progresso, cronômetro e animação final mostrando o caminho encontrado pela IA.

2.6 ADAPTAÇÃO DAS ESTRUTURAS DE DADOS

As bibliotecas de Pilha e Fila fornecidas inicialmente eram projetadas para manipular tipos primitivos simples, como inteiros. Entretanto, o problema do *8-Puzzle* exige uma representação mais completa dos estados explorados. A dificuldade residiu no fato de que cada nó necessita armazenar a matriz 3×3 , profundidade e histórico de movimentos. A solução foi a criação do arquivo TIPO.h, definindo o tipo composto Estado. As bibliotecas PILHA.h e FILA.h foram refatoradas para manipular esse tipo complexo, permitindo transportar todas as informações necessárias à reconstrução do caminho da solução.

2.7 PORTABILIDADE E COMPATIBILIDADE (CROSS-PLATFORM)

Durante o desenvolvimento, surgiu a necessidade de utilizar funções de sistema para limpar a tela e pausar a execução, mas essas funções variam entre sistemas operacionais. A dificuldade era que `Sleep()` e `system('cls')` funcionam apenas no Windows, enquanto `usleep()` e `system('clear')` são usados em Unix/Linux e macOS. Implementaram-se diretivas de pré-processamento (`#ifdef _WIN32`) no arquivo `FuncoesGerais.c`, permitindo ao código identificar o sistema operacional e selecionar automaticamente as bibliotecas adequadas (`windows.h` ou `unistd.h`). Essa abordagem garante que o jogo funcione corretamente em qualquer ambiente.

3. CONSIDERAÇÕES FINAIS

O desenvolvimento do projeto permitiu a aplicação prática de conceitos fundamentais da Inteligência Artificial, em especial os algoritmos de busca em grafos. A equipe enfrentou desafios importantes relacionados ao consumo de memória, otimização das buscas e criação de interface, alcançando uma solução funcional e informativa. O projeto foi um exercício valioso na aplicação de estruturas de dados complexas para resolver problemas de otimização de busca.

REFERÊNCIAS

CANVAS. Materiais da disciplina disponibilizados pelo Professor Denis. Acesso em: 17 nov. 2025.

GEEKSFORGEEEKS. *8 Puzzle Problem Explanation and Solutions*. Disponível em: www.geeksforgeeks.org. Acesso em: 18 nov. 2025.

RUSSELL, Stuart; NORVIG, Peter. *Inteligência Artificial: Uma Abordagem Moderna*. 3. ed. Rio de Janeiro: Elsevier, 2013.